Lab Assignment 6: Source Code Analyzer Using Lex

Course: CS3104 - Compiler Design Laboratory

Date: 09/09/2026

Instructions

- Write your name and roll number on top of the. l source file as comments.
- Source files must be named exactly: `lexer.l` The program must compile cleanly with plain gcc, e.g.: .
- The program should compile and run using standard Lex/Flex commands, for example:

```
flex lexer.l
gcc lex.yy.c -o lexer
```

- Submit only the source file through the same submission portal used for previous assignments.
  http://10.22.11.15/~abyaym/2026/CS3104_26/submission/

Problem Statement

In the previous lab, you used Lex to recognize individual tokens such as identifiers, constants, operators, and keywords. In this assignment, you will extend the lexical analysis process to **analyze a source program at a higher level**. Your task is to build a Lex-based **Source Code Analyzer** that reads a C-like source file and:

1. Identifies and removes comments.
2. Recognizes string and character literals as complete lexemes.
3. Recognizes preprocessor directives.
4. Detects identifiers appearing in the source program.
5. Maintains statistics about the different lexical constructs encountered.
6. Reports lexical errors for unterminated comments, strings, or character literals.

   The purpose of this assignment is to understand how Lex can be used not only to print tokens, but also to **process and summarize a source program**

Lexical Constructs to Recognize

1. Single-line comments

A comment beginning with:

```
//
```

continues until the end of the line.

Example

```
x = x + 1; // increment x
```

The entire comment should be ignored by the analyzer.

2. Multi-line comments

Comments enclosed between:

```
/*
...
*/
```

Must be treated as a single comment

3. String literals

A string enclosed in double quotes should be recognized as **one complete lexeme**.
Example:

```
"Hello World"
"Enter your name:"
"Value = "
```

The output should contain:

```
STRING_LITERAL    "Hello World"
```

4. Character Literals

A character enclosed in single quotes should be recognized as one lexeme.

Examples:

```
'a'
'Z'
'5'
```

Output:

```
CHAR  LITERAL      'a'
```

5. Preprocessor directives

Recognize lines beginning with # as preprocessor directives.

For Example:

```
PREPROCESSOR      #include <stdio.h>
PREPROCESSOR      #define MAX 100
```

6. Identifiers

Example:

```
total
count
student_name
calculate
```

Each occurrence must be reported

7. Other characters

Characters that do not belong to comments, literals, preprocessor directives, or identifiers may be treated as ordinary source characters and need not be printed individually.However, the analyzer should keep track of the number of lines processed.

Example

Input

```
#include <stdio.h>

int main() {
    // Read the value
    int count = 10;

    printf("Count = %d\n", count);

    /*
       Update count
       before returning.
    */
    count = count + 1;

    return 0;
}
```

Expected Output:

```
PREPROCESSOR      #include <stdio.h>

IDENTIFIER        main
IDENTIFIER        count
STRING_LITERAL    "Count = %d\n"
IDENTIFIER        count
IDENTIFIER        count
IDENTIFIER        count
```

```
IDENTIFIER       return

------------------------------
Source Code Statistics
------------------------------
Lines            : 16
Comments         : 2
String literals  : 1
Character literals: 0
Preprocessor     : 1
Identifiers      : 6
------------------------------
ACCEPT
```

Core Requirements

● **Comment recognition:** recognize both `//` and `/* ... */` comments using Lex rules.

● **Comment removal:** comments must not be reported as identifiers or other lexical constructs.

● **String recognition:** recognize a complete string literal as one lexeme.

For example:

```
"hello world"
```

must not be split into separate words.

● **Character recognition:** recognize character literals including common escaped characters such as:

```
'\n'
'\t'
'\\'
'\''
```

● **Preprocessor recognition:** recognize a complete preprocessor directive beginning with #.

● **Identifier counting:** print every identifier encountered and maintain a count of the total number of identifier occurrences.

● **Line counting:** maintain the total number of lines in the input.

● **Comment counting:** maintain separate counts for:

```
single-line comments
multi-line comments
```

● **Literal counting:** maintain separate counts for:

```
string literals
character literals
```

● **Preprocessor counting:** maintain the total number of preprocessor directives.

● **Lexical error detection:** detect at least the following malformed constructs:

```
unterminated multi-line comment
unterminated string literal
unterminated character literal
```

● **Final verdict:** if no lexical error occurs, the program must end with:

ACCEPT

Otherwise, it must end with:

REJECT

Implementation Guidelines

● Use the **definitions section** of Lex to define reusable regular-expression components where appropriate.

● Use Lex **start conditions** if necessary to handle multi-line comments or other constructs that require multiple scanning states.

For example, a comment may begin with:

```

```

and continue across several lines until:

```

```

is encountered.

● A string literal should be recognized as one unit even if it contains spaces.

For example:

```

```

is one `STRING_LITERAL`.

● Take care when processing escaped characters inside literals.

For example:

```

```

should be treated as one string literal.

● A `//` appearing inside a string literal must **not** start a comment.

For example:

```

```

is a string literal, not a comment.

● Similarly, `/*` appearing inside a string literal must not start a multi-line comment.

● A `#` appearing inside a string literal must not be interpreted as a preprocessor directive.

● Keep separate counters for each required category rather than deriving the statistics afterward.

● Use `yytext` to access the matched lexeme and `yylineno` or an equivalent mechanism for line tracking.

● The generated `lex.yy.c` file must not be submitted.

Compilation & Testing

Compile Using:

```
$ flex lexer.l
$ gcc lex.yy.c -o lexer
$ ./lexer < input.c
```

The submitted program should be tested with:

1. Single-line comments
2. Multi-line comments

3. Strings containing spaces
4. Strings containing comment-like characters
5. Character literals
6. Escaped characters
7. Preprocessor directives
8. Unterminated strings
9. Unterminated character literals
10. Unterminated comments
11. Multiple constructs occurring together

Deliverables

- `lexer.l`

Only the. l file should be submitted.